SECURE GATEWAY SETUP

Write-Up

by Raphael Beauregard

[MAKING SURE THE LABS ENV. IS SECURE]

by adding an IDS & Firewall rules

23 APRIL 2026

Table of Contents

Goal.....3

Objective.....3
Considerations.....3
Expectations.....3

Methodology.....4

Tools.....4

Execution.....5

Creation of new LANs.....5
Setting up the routing.....7
Snort Configuration.....10
Configuring an extra Firewall.....13
Choosing the firewall.....13

Conclusion.....14

Goal

Objective

The objective of this venture is to route all traffic through one machine. This machine can inspect packets, block/allow them, and generate alerts based on a set of rules. Our security gateway will act as a guard between not only the LAN and the internet, but also as a shield for each client against other clients.

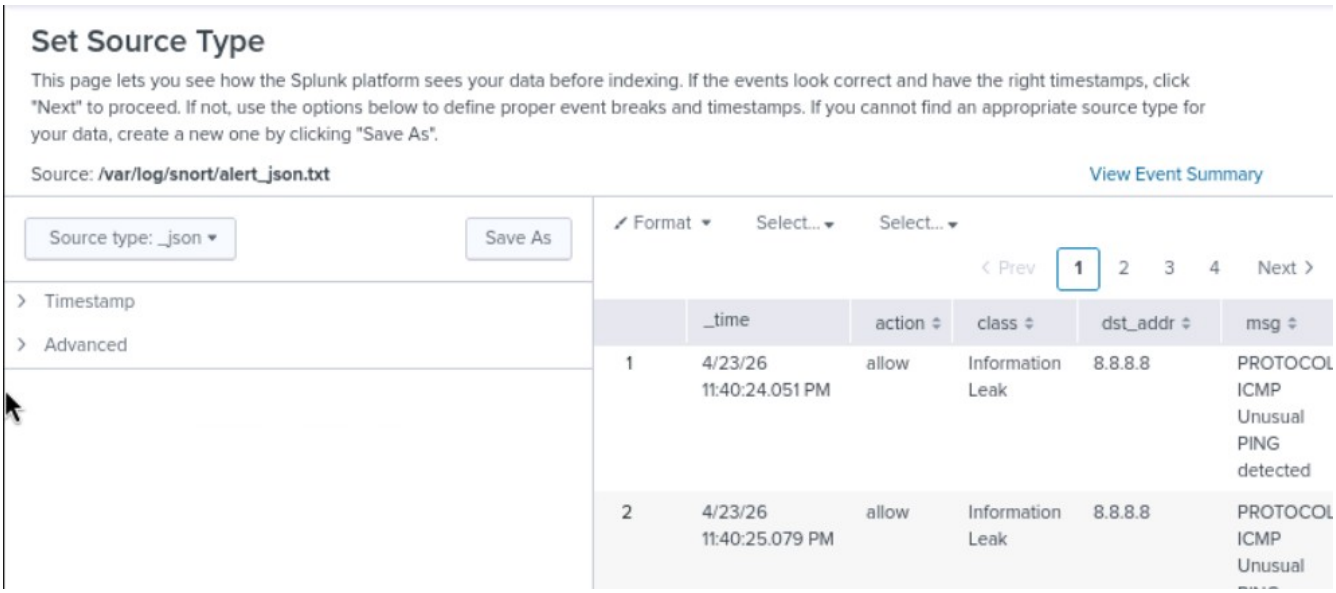
Considerations

Since I do not have infinite resources, I have to set up this on the same machine as the SIEM. This means one machine I will call a security server will act as a log aggregator + Gateway + IDS. In a real scalable environment, I would do things differently for performance purposes, but since there will never be more than 5-10 clients on here, I can just adjust the CPU and RAM allocation of the VM to fit needs as we go.

Expectations

I expect things to be easy enough, as I have experience with routing in Linux. I have also set up a basic pfSense instance in the past, which is similar to this LAB, just without the SIEM and the IDS.

Methodology



Tools

- Splunk
- Snort
- virsh
- QEMU/KVM
- nftables

I am going to create a seperate internal LAN interface that has no bridge to the host server (the ones running the VMs). It will be fully isolated. The gateway will point to the security server. The security server will have two interfaces: the isolated LAN and another network that will be bridged for Internet access. Even if a

client changes their configuration, it will not be able to access the Internet unless it passes through the security server.

I will also have to add the logs of Snort and nftables (or global networking logs) as data sources in Splunk.

Execution

Creation of new LANs

```
  GNU nano 8.5                                        isolated_net.xml
<network>
  <name>isolated_splunk_lan</name>
  <bridge name="virbr_ispl"/>
  <ip address="192.168.15.1" netmask="255.255.255.0">
    <dhcp>
      <range start="192.168.15.20" end="192.168.15.200"/>
    </dhcp>
  </ip>
</network>
```

Figure 1: Creation of XML to define the network

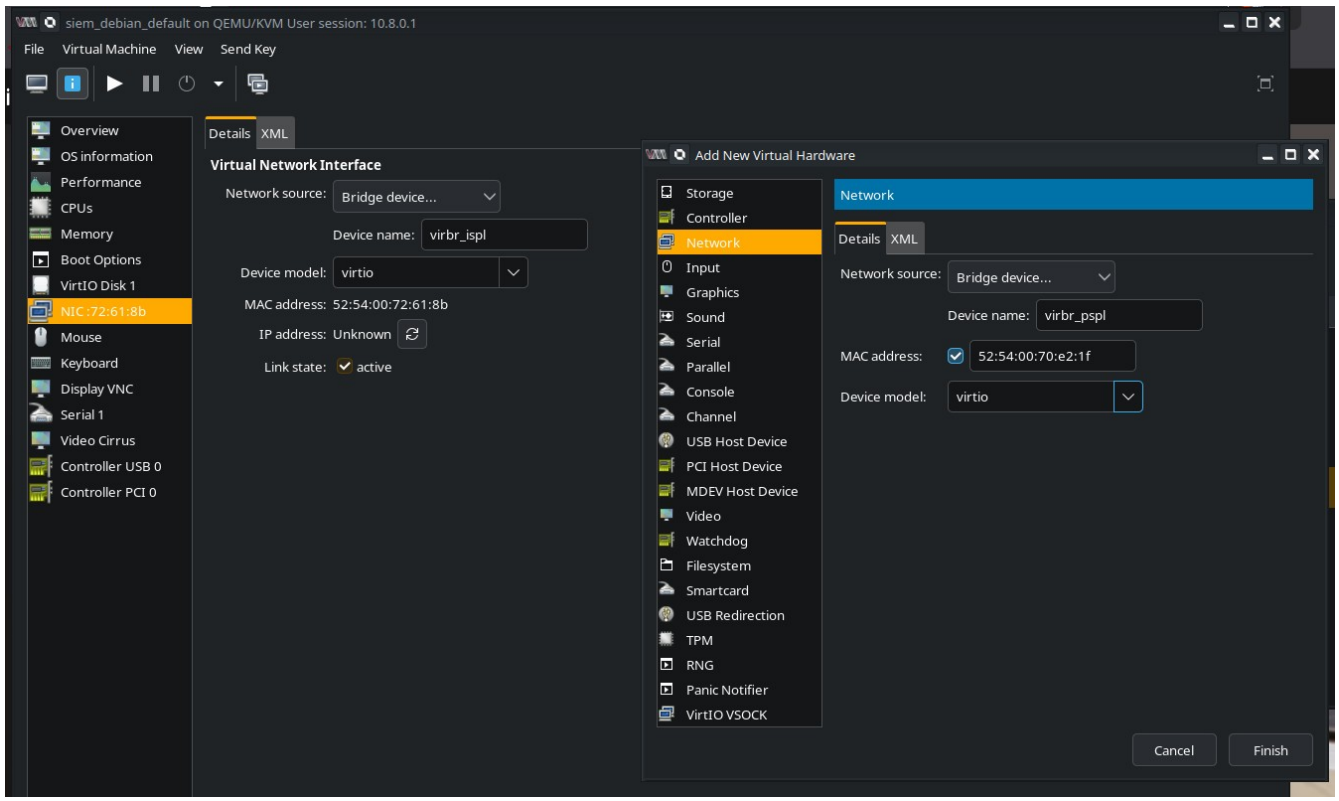
As I was setting this up, I've also decided to delete the original splunk_lan I had to use a more distinct IP range.

```
  GNU nano 8.5                                        network.xml
<network>
  <name>public_splunk_lan</name>
  <forward mode="nat"/>
  <bridge name="virbr_pspl" stp="on" delay="0"/>
  <ip address="10.4.0.1" netmask="255.255.255.0">
    <dhcp>
      <range start="10.4.0.10" end="10.4.0.100"/>
    </dhcp>
  </ip>
</network>
```

I also have to allow the bridge to be created by QEMU.

```
  GNU nano 8.5                    /etc/qemu/bridge.conf
allow virbr0
allow virbr2
allow virbr_ispl
allow virbr_pspl
allow eno1
```

Now, we have to create a new network card for the security server, and change the existing one.



For every client, I assigned their only network interfaces to *virbr_ispl* which is the isolated LAN.

Setting up the routing

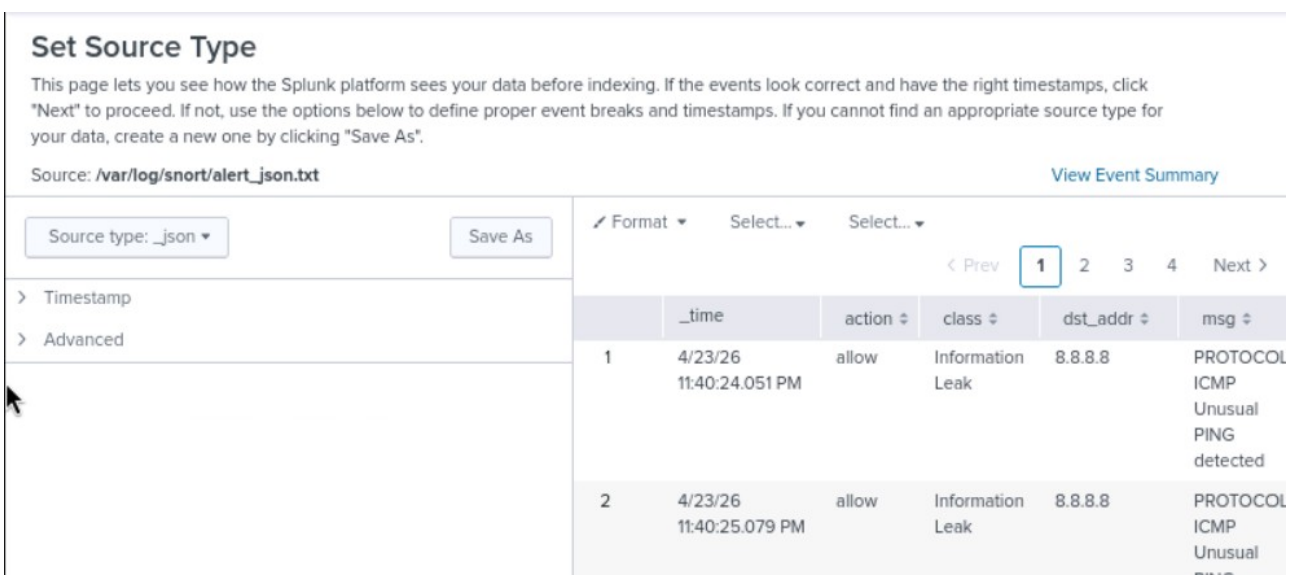
So now that we have both interfaces with ready, with eth0 having the static IP of the default gateway...

```
root@trixie:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 52:54:00:72:61:8b brd ff:ff:ff:ff:ff:ff
    altname enp0s5
    altname ens5
    altname enx52540072618b
    inet 192.168.15.1/24 scope global eth0
       valid_lft forever preferred_lft forever
    inet6 fe80::5054:ff:fe72:618b/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 52:54:00:70:e2:1f brd ff:ff:ff:ff:ff:ff
    altname enp0s6
    altname ens6
    altname enx52540070e21f
    inet 10.4.0.74/24 brd 10.4.0.255 scope global dynamic eth1
       valid_lft 3532sec preferred_lft 3532sec
    inet6 fe80::5054:ff:fe70:e21f/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
```

We can start routing and forward traffic.

```
root@trixie:~# sudo sysctl -w net.ipv4.ip_forward=1
net.ipv4.ip_forward = 1
root@trixie:~# sudo iptables -t nat -A POSTROUTING -o eth1 -j MASQUERADE
root@trixie:~# sudo iptables -A FORWARD -i eth0 -o eth1 -j ACCEPT
root@trixie:~# sudo iptables -A FORWARD -i eth1 -o eth0 -m state --state ESTABLISHED,RELATED -j ACCEPT
```

- We enable IP forwarding



- We hide the original source IP with NAT

- We allow forwards from eth0 to eth1
- We allow forwards from eth1 to eth0 only if there’s an existing relationship for the packet

These rules are, by default, temporary. However, you can use the iptables-persistent package to save your configuration.

```
Processing triggers for man-db (2.13.1-1) ...
root@trixie:~# sudo netfilter-persistent save
run-parts: executing /usr/share/netfilter-persistent/plugins.d/15-ip4tables save
run-parts: executing /usr/share/netfilter-persistent/plugins.d/25-ip6tables save
root@trixie:~# _
```

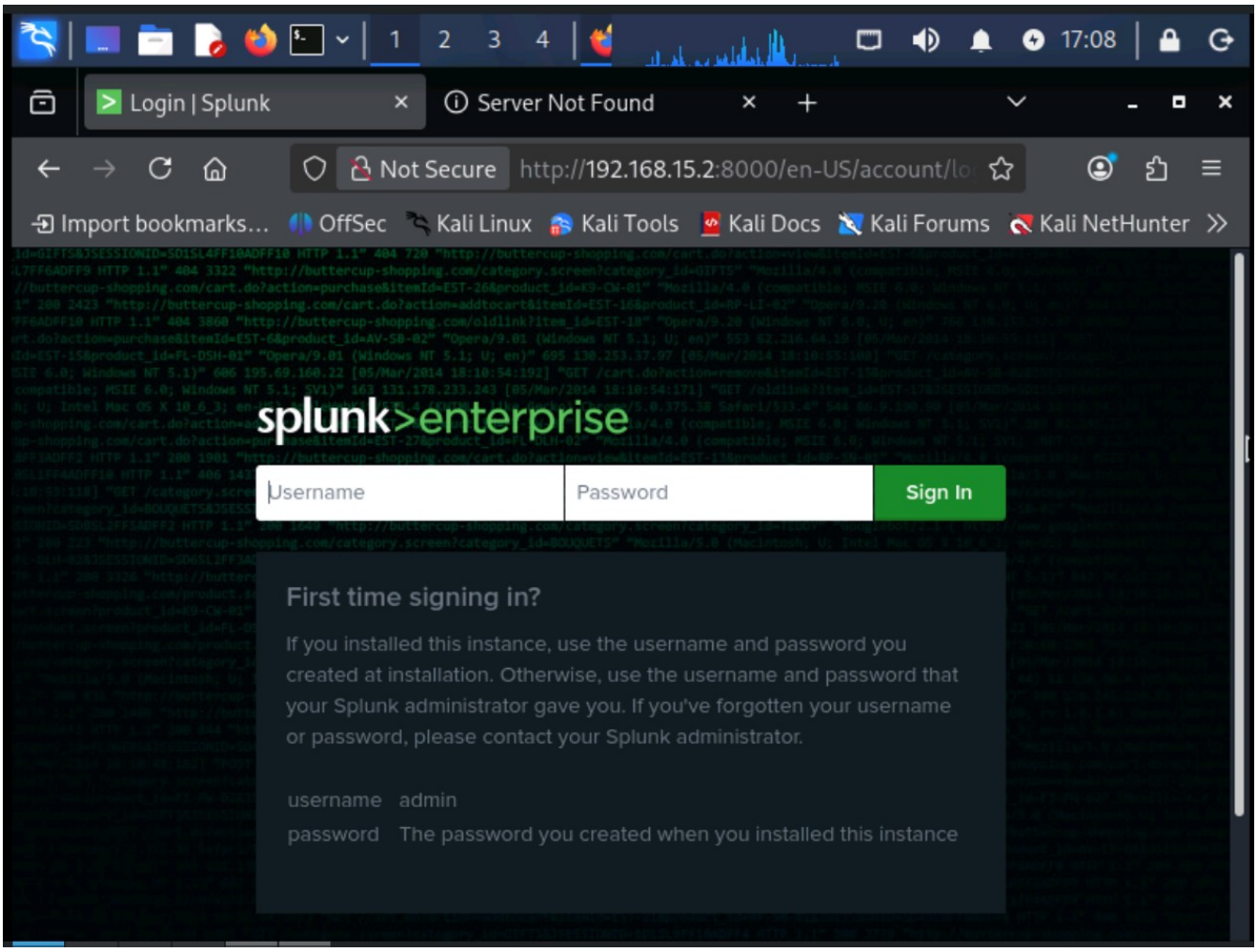
Since I have not created a DHCP server, and I do not want to fight with XML Network creation stuff, I will set static IP confgurations, DNS settings, etc... for the clients. Once that’s done, they can access the Internet. Consider DNS and DHCP server future

```
vagrant@vagrant:~$ ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=114 time=6.33 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=114 time=5.03 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=114 time=4.65 ms
^C
--- 8.8.8.8 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 4.647/5.334/6.330/0.720 ms
vagrant@vagrant:~$
```

Figure 2: Ubuntu Service Server with Internet access

I ran into some issues afterwards because the virtual network gateway also uses 192.168.15.1. Finding the source of some issues took me a while. I changed it to .2 to avoid conflicts.

Net.ipv4.ip_forward also resets itself after a reboot, so I had to add it to /etc/systctl.conf. LAN devices also still have access to local services, of course.



Snort Configuration

After building Snort 3 and installing its dependencies (libdaq and other libraries), it was time to configure it. I first created the necessary directories.

```
        snort [-options] -c conf -i iface: process live
        snort [-options] -c conf -r pcap: process readback
root@trixie:~# sudo mkdir -p /usr/local/snort/etc/snort/
root@trixie:~# sudo mkdir -p /usr/local/snort/rules
root@trixie:~# sudo mkdir -p /var/log/snort
root@trixie:~# sudo chmod 755 /var/log/snort
root@trixie:~#
```

Then, I set up the lua configuration under /usr/local/snort/etc/snort/snort.lua

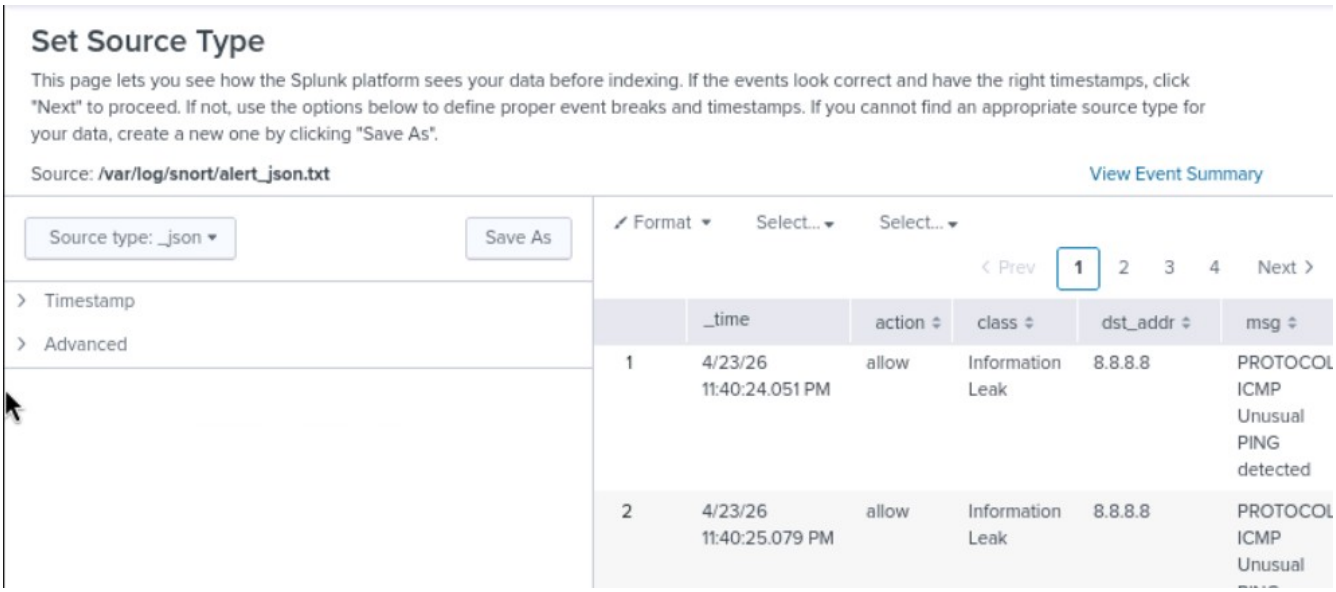
```
-------------------------------------------------------------------------------
-- 1. configure defaults
-------------------------------------------------------------------------------

-- HOME_NET and EXTERNAL_NET must be set now
-- setup the network addresses you are protecting
HOME_NET = '192.168.15.0/24'

-- set up the external network addresses.
-- (leave as "any" in most situations)
EXTERNAL_NET = '!$HOME_NET'

include 'snort_defaults.lua'
-
-------------------------------------------------------------------------------
```

I will only run Snort in IDS mode, because I want to use this environment to learn attack patterns later on (blocking the attack altogether would not allow me to study them).



Next up, we have to ensure Snort outputs in JSON so that Splunk can easily parse the data and also make Snort act as an IDS, not an IPS in the configuration file.

After this, all that’s left for the initial configuration is to make Snort run on boot with a systemd service. I added some arguments to Snort because I had issues with the config file.

```
root@trixie:/tmp/libdaq# sudo systemctl daemon-reload
root@trixie:/tmp/libdaq# sudo systemctl enable snort
Created symlink '/etc/systemd/system/multi-user.target.wants/snort.service' → '/etc/systemd/system/snort.service'.
root@trixie:/tmp/libdaq# sudo systemctl start snort
root@trixie:/tmp/libdaq# sudo systemctl status snort
● snort.service - Snort 3 Network IDS
     Loaded: loaded (/etc/systemd/system/snort.service; enabled; preset: enabled)
     Active: activating (auto-restart) since Thu 2026-04-23 23:04:46 UTC; 3s ago
 Invocation: 69c30bb87ca046cfb44f5dd8d9bfa531
    Process: 8620 ExecStartPre=/bin/mkdir -p /var/log/snort (code=exited, status=0/SUCCESS)
    Process: 8621 ExecStartPre=/bin/chmod 755 /var/log/snort (code=exited, status=0/SUCCESS)
    Process: 8624 ExecStart=/usr/local/snort/bin/snort -c /usr/local/snort/etc/snort/snort.lua -D (code=exited, status=0/SUCCES>
   Main PID: 8624 (code=exited, status=0/SUCCESS)
   Mem peak: 48.9M
        CPU: 251ms

Apr 23 23:04:46 trixie systemd[1]: snort.service: Deactivated successfully.
lines 1-12/12 (END)
```

...and here, we now have logs for unusual activity.

```
root@trixie:/tmp/libdaq# cat /var/log/snort/alert_fast.txt
04/23-23:40:24.051692 [**] [1:29456:3] "PROTOCOL-ICMP Unusual PING detected" [**] [Classification: Information Leak] [Priority:
2] {ICMP} 192.168.15.5 -> 8.8.8.8
04/23-23:40:25.079321 [**] [1:29456:3] "PROTOCOL-ICMP Unusual PING detected" [**] [Classification: Information Leak] [Priority:
2] {ICMP} 192.168.15.5 -> 8.8.8.8
04/23-23:42:12.982114 [**] [1:29456:3] "PROTOCOL-ICMP Unusual PING detected" [**] [Classification: Information Leak] [Priority:
2] {ICMP} 192.168.15.5 -> 8.8.8.8
04/23-23:42:12.987497 [**] [1:408:8] "PROTOCOL-ICMP Echo Reply" [**] [Classification: Misc activity] [Priority: 3] {ICMP} 8.8.8
8 -> 192.168.15.5
root@trixie:/tmp/libdaq#
```

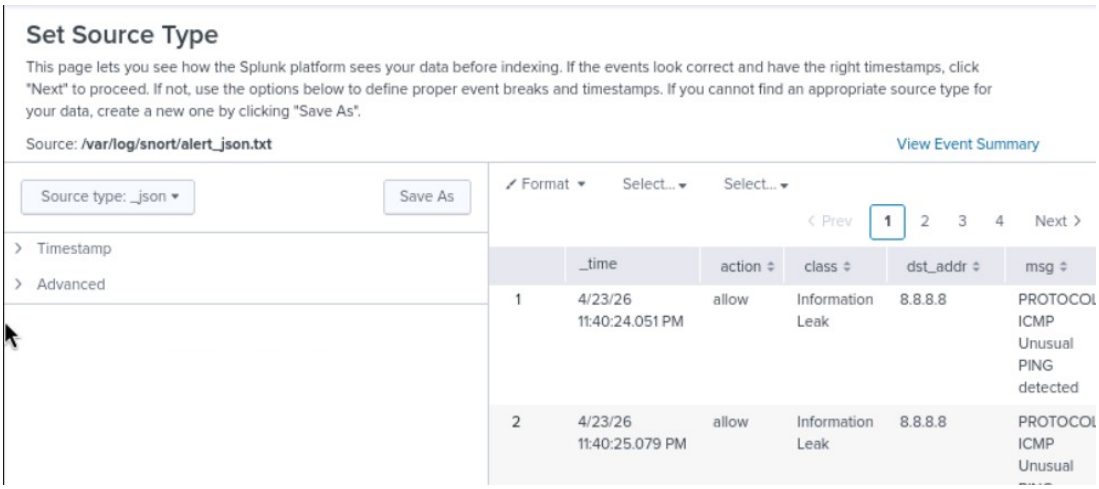
I've added a custom test rule as well:

```
  GNU nano 8.4                    /usr/local/snort/rules/local.rules
#Alert on any ICMP between LAN hosts
alert icmp $HOME_NET any -> $HOME_NET any (msg:"LAN host pinging another LAN host"; sid:1000001; rev:1;)

# Alert on ICMP from LAN to anywhere
alert icmp $HOME_NET any -> $EXTERNAL_NET any (msg:"LAN host pinging external"; sid:1000002; rev:1;)
```

```
5.2
04/23-23:54:19.121463 [**] [1:1000001:1] "LAN host pinging another LAN host" [**] [Priority: 0] {ICMP} 192.168.15.2 -> 192.168.1
5.5
04/23-23:54:20.145578 [**] [1:1000001:1] "LAN host pinging another LAN host" [**] [Priority: 0] {ICMP} 192.168.15.5 -> 192.168.1
5.2
04/23-23:54:20.145602 [**] [1:1000001:1] "LAN host pinging another LAN host" [**] [Priority: 0] {ICMP} 192.168.15.2 -> 192.168.1
5.5
04/23-23:54:21.169407 [**] [1:1000001:1] "LAN host pinging another LAN host" [**] [Priority: 0] {ICMP} 192.168.15.5 -> 192.168.1
5.2
04/23-23:54:21.169432 [**] [1:1000001:1] "LAN host pinging another LAN host" [**] [Priority: 0] {ICMP} 192.168.15.2 -> 192.168.1
5.5
```

We can now add the JSON version as a data source in Splunk.



Configuring an extra Firewall

I could use Snort to black some traffic, but that would be too easy. Using another application will also allow me to just turn that firewall off to try pen-testing things, but still get my Splunk logs due to Snort still being active. I want to familiarize myself with another tool too.

Choosing the firewall

I decided to use UFW, as it is native to the Debian ecosystem. It has simple and intuitive syntax, which will reduce chances of accidental misconfiguration. In a more complex environment, I might go for firewall-cmd instead.

UFW is very easy to install through APT, and enabling it is as simple as one command.

```
root@trixie:/tmp/libdaq# ufw enable
Firewall is active and enabled on system startup
root@trixie:/tmp/libdaq# ufw status
Status: active
```

Now, I will add a rule to block SSH traffic over port 22 from the internet.

```
root@trixie:/tmp/libdaq# sudo ufw deny in on eth1 to any port 22
Rule added
Rule added (v6)
```

```
root@trixie:/tmp/libdaq# sudo ufw allow 22
Rule updated
Rule updated (v6)
```

```
root@trixie:/tmp/libdaq# ufw status
Status: active

To                         Action      From
--                         ------      ----
22 on eth1                 DENY        Anywhere
22                         ALLOW       Anywhere
22 (v6) on eth1            DENY        Anywhere (v6)
22 (v6)                    ALLOW       Anywhere (v6)
```

I’m going to add rules as time goes on, and see what kind of rules could prevent certain attacks from passing through. I can also add UFW logs to Splunk!

Conclusion

...and there you have it! The SIEM is actively aggregating logs from clients and itself. The hard (or more so boring) part of setup is done. I can now attack my own machines, and defend against those attacks using the logs as threat intelligence.